PMSCS 622P
Programming Languages

Lecture 07: Data Structures

Mr. Md. Masum Bhuiyan

Department of Computer Science and Engineering
Jahangirnagar University

Lecture Roadmap

1. Lists (Dynamic Arrays)
2. Linked Lists
3. Stacks
4. Queues
5. Dictionaries (Hash Maps)
6. Sets (Hash Sets)

What is a Data Structure?

A **Data Structure** is a specialized format for organizing, processing, retrieving, and storing data in computer memory.

Different tasks require different structures. Choosing the right data structure determines the time and space complexity of your algorithm.

In this module, we will explore the core structures, understand their performance trade-offs, and implement algorithmic solutions using precise code.

1.1 Concept: Lists

A **List** acts as a Dynamic Array. It stores elements in contiguous memory blocks, allowing for instant mathematical indexing.

1.2 Operations & Complexity

Lists provide instant access but suffer when elements need to be shifted.

Operation	Time Complexity
Access by Index	$O(1)$
Append to end	$O(1)$ (Amortized)
Pop from end	$O(1)$
Insert in middle/front	$O(N)$
Search by value	$O(N)$

1.3 Dynamic Resizing

Because they are contiguous, lists cannot grow infinitely in their current memory slot.

When a list reaches its capacity limit:

1. A new, larger contiguous block of memory is allocated.
2. All existing elements are copied to the new block ($O(N)$ operation).
3. The old memory block is freed.

This is why appending is $O(1)$ *amortized*the $O(N)$ resize happens rarely.

Problem 1.1: Find the Maximum Element

Problem: Given a list of unsorted integers, find the largest number without using built-in functions.

Idea: Initialize the max value with the first element. Iterate through the array and update the max value whenever a larger number is encountered.

Code:

```
def find_maximum(arr):
    if not arr:
        return None

    current_max = arr[0]

    for number in arr:
        if number > current_max:
            current_max = number

    return current_max
```

Problem 1.2: Reverse a List In-Place

Problem: Reverse the elements of an array without allocating a new array.

Idea: Use two pointers, one at the start and one at the end. Swap their values and move them towards the center until they meet.

Code:

```
def reverse_array(arr):
    left = 0
    right = len(arr) - 1

    while left < right:
        arr[left], arr[right] = arr[right], arr[left]
        left += 1
        right -= 1

    return arr
```

Problem 1.3: Move Zeroes to End

Problem: Move all 0's to the end of an array while maintaining the relative order of the non-zero elements.

Idea: Iterate through the array, placing all non-zero elements sequentially at the front. Fill the remaining positions with zeros.

Code:

```
def move_zeroes(arr):
    insert_pos = 0

    for i in range(len(arr)):
        if arr[i] != 0:
            arr[insert_pos] = arr[i]
            insert_pos += 1

    while insert_pos < len(arr):
        arr[insert_pos] = 0
        insert_pos += 1

    return arr
```

2.1 Linked Lists

A **Linked List** allocates memory dynamically for each element (a Node). Elements are not contiguous; they are linked via pointers.

2.2 Implementation Structure

Because pointers must be manually managed, we construct Linked Lists using OOP classes.

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class LinkedList:
    def __init__(self):
        self.head = None
```

2.3 Operations & Trade-offs

Linked Lists trade access speed for insertion speed.

Operation	Time Complexity
Access by Index	$O(N)$
Insert at Head	$O(1)$
Delete at Head	$O(1)$
Search by value	$O(N)$

Problem 2.1: Traverse and Print

Problem: Traverse a Singly Linked List and print every element.

Idea: Start a pointer at the head node. In a loop, print the data and advance the pointer to the next node until the pointer becomes ‘None‘.

Code:

```
def print_linked_list(head):
    current = head

    while current is not None:
        print(current.data)
        current = current.next
```

Problem 2.2: Search for a Value

Problem: Given the head of a list, return True if a target value exists, otherwise False.

Idea: Walk the list node by node. If the current node's data matches the target, return True. If the loop finishes without returning, return False.

Code:

```
def search_list(head, target):
    current = head

    while current is not None:
        if current.data == target:
            return True
        current = current.next

    return False
```

Problem 2.3: Insert at the Beginning

Problem: Given a Linked List and a new value, insert the value as the new Head.

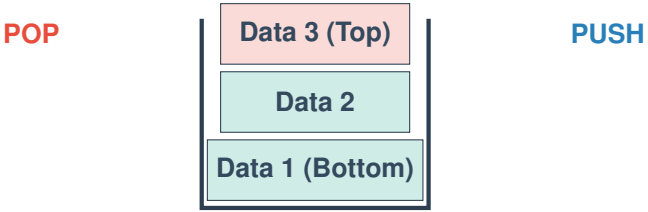
Idea: Instantiate a new Node. Point the new node's 'next' reference to the current head of the list. Update the list's head reference to the new node.

Code:

```
def insert_at_head(linked_list, new_value):
    new_node = Node(new_value)
    new_node.next = linked_list.head
    linked_list.head = new_node
```

3.1 Stacks

A **Stack** is a restricted linear data structure enforcing the **Last-In, First-Out (LIFO)** rule. Elements are added to the top and removed from the top.



3.2 Operations & Complexity

Stacks restrict access to ensure $O(1)$ operations on the active end.

Operation	Time Complexity
Push (Add to top)	$O(1)$
Pop (Remove top)	$O(1)$
Peek (View top)	$O(1)$
Check if Empty	$O(1)$

3.3 Array-Based Implementation

A Stack is conceptually an Abstract Data Type (ADT). It can be implemented using a standard List by artificially restricting usage to ‘append‘ and ‘pop‘.

```
class Stack:
    def __init__(self):
        self.items = []

    def push(self, item):
        self.items.append(item)

    def pop(self):
        return self.items.pop()

    def peek(self):
        return self.items[-1]
```

Problem 3.1: Valid Parentheses

Problem: Given a string of brackets '()[]' determine if the sequence is valid.

Idea: Push open brackets onto a stack. For closing brackets, pop the stack and verify it matches the corresponding open bracket.

Code:

```
def is_valid(s):
    stack = []
    pairs = {')': '(', '}': '{', ']': '['}

    for char in s:
        if char in pairs.values():
            stack.append(char)
        elif char in pairs.keys():
            if not stack or stack.pop() != pairs[char]:
                return False

    return len(stack) == 0
```

Problem 3.2: Reverse a String

Problem: Reverse a string utilizing a Stack data structure.

Idea: Push every character of the string onto a stack. Pop them off one by one to construct the reversed string.

Code:

```
def reverse_string_with_stack(s):
    stack = []

    for char in s:
        stack.append(char)

    reversed_str = ""

    while stack:
        reversed_str += stack.pop()

    return reversed_str
```

Problem 3.3: Decimal to Binary Conversion

Problem: Convert a Base-10 integer to a Base-2 binary string using a Stack.

Idea: Successively divide the number by 2, pushing the remainders onto a stack. Pop the stack to build the binary string.

Code:

```
def decimal_to_binary(dec_number):
    if dec_number == 0:
        return "0"

    stack = []

    while dec_number > 0:
        stack.append(dec_number % 2)
        dec_number = dec_number // 2

    binary_str = ""
    while stack:
        binary_str += str(stack.pop())

    return binary_str
```

4.1 Queues

A **Queue** enforces the **First-In, First-Out (FIFO)** rule. Elements are added to the rear and removed from the front.

4.2 Operations & Complexity

Using a standard Array for a Queue is disastrous ($O(N)$ Dequeue). A proper queue implementation guarantees $O(1)$ operations on both ends.

Operation	Time Complexity
Enqueue (Add to rear)	$O(1)$
Dequeue (Remove front)	$O(1)$
Peek (View front)	$O(1)$

4.3 Implementation: The Deque

To achieve $O(1)$ performance, we use a Double-Ended Queue (`deque`), which is internally backed by a doubly-linked list.

```
from collections import deque

q = deque()
q.append("Task 1")      # Enqueue
q.append("Task 2")
front_task = q.popleft() # Dequeue
```

Problem 4.1: Print Queue Simulation

Problem: Simulate a print queue that processes jobs in the order they arrive.

Idea: Enqueue all incoming documents into a deque. Loop through the deque, dequeuing and printing documents until it is empty.

Code:

```
from collections import deque

def printer_simulation(document_list):
    printer_queue = deque()

    for doc in document_list:
        printer_queue.append(doc)

    while printer_queue:
        current_job = printer_queue.popleft()
        print(current_job)
```

Problem 4.2: Reverse First K Elements

Problem: Given a queue and an integer K, reverse the first K elements.

Idea: Dequeue K elements into a stack. Pop the stack and enqueue them back to the queue. Then, rotate the remaining $N - K$ elements to the back.

Code:

```
def reverse_k_elements(queue, k):
    if not queue or k > len(queue):
        return queue

    stack = []

    for _ in range(k):
        stack.append(queue.popleft())

    while stack:
        queue.append(stack.pop())

    for _ in range(len(queue) - k):
        queue.append(queue.popleft())

    return queue
```

Problem 4.3: Palindrome Checker

Problem: Use a double-ended queue to check if a word is a palindrome.

Idea: Load characters into a deque. Continuously pop from both the front and the rear, comparing them. If they mismatch, it's not a palindrome.

Code:

```
from collections import deque

def check_palindrome(word):
    char_deque = deque(word)

    while len(char_deque) > 1:
        front = char_deque.popleft()
        back = char_deque.pop()

        if front != back:
            return False

    return True
```

5.1 Dictionaries

A **Dictionary** stores data in Key-Value pairs. It uses a Hash Function to map Keys directly to memory addresses, bypassing sequential search.

- **Keys:** Must be unique and immutable (Strings, Integers, Tuples).
- **Values:** Can be any data type.

5.2 Operations & Complexity

Dictionaries sacrifice memory space to achieve constant time lookups.

Operation	Time Complexity
Access by Key	$O(1)$ (Average)
Insert / Update	$O(1)$ (Average)
Delete	$O(1)$ (Average)
Search (if key exists)	$O(1)$ (Average)

5.3 Hash Functions and Collisions

Behind the scenes, a Hash Function transforms a string into an integer array index.

If two keys resolve to the same index, a **Collision** occurs. This is resolved internally (via open addressing in modern dictionaries), ensuring performance degrades gracefully only in worst-case scenarios.

```
data = {}
data["Apple"] = 5
data["Banana"] = 8
val = data["Apple"]
```

Problem 5.1: Word Frequency Counter

Problem: Given a string of text, count the occurrences of each word.

Idea: Split the string. Use a dictionary where the keys are words and values are the running counts. Increment the count if the word is seen.

Code:

```
def count_words(text):
    words = text.split()
    freq_map = {}

    for word in words:
        if word in freq_map:
            freq_map[word] += 1
        else:
            freq_map[word] = 1

    return freq_map
```

Problem 5.2: Two Sum

Problem: Return the indices of the two numbers in an array that add up to a specific target.

Idea: Iterate through the array. Store elements and their indices in a dictionary. For each element, check if its complement (‘target - element‘) exists in the dictionary.

Code:

```
def two_sum(arr, target):
    seen = {}

    for i in range(len(arr)):
        current = arr[i]
        complement = target - current

        if complement in seen:
            return [seen[complement], i]

        seen[current] = i

    return None
```

Problem 5.3: First Non-Repeating Character

Problem: Find the first non-repeating character in a string and return its index.

Idea: Make one pass to count character frequencies in a dictionary. Make a second pass through the string to find the first character with a count of 1.

Code:

```
def first_unique_char(s):
    counts = {}

    for char in s:
        if char in counts:
            counts[char] += 1
        else:
            counts[char] = 1

    for i in range(len(s)):
        if counts[s[i]] == 1:
            return i

    return -1
```

6.1 Sets

A **Set** is implemented as a Hash Table, but it only contains Keys (no Values).

- **Uniqueness:** All elements are mathematically unique.
- **Unordered:** Elements have no index.
- **Performance:** Existence checks (‘x in set‘) are extremely fast.

6.2 Mathematical Set Operations

Sets excel at comparing groups of data using underlying mathematical logic.

```
A = {1, 2, 3}
B = {3, 4, 5}

union_set = A | B          # {1, 2, 3, 4, 5}
intersect_set = A & B      # {3}
diff_set = A - B           # {1, 2}
```

6.3 Operations & Complexity

Like dictionaries, sets optimize for $O(1)$ logic operations.

Operation	Time Complexity
Add element	$O(1)$ (Average)
Remove element	$O(1)$ (Average)
Search (if exists)	$O(1)$ (Average)
Union / Intersection	$O(N)$

Problem 6.1: Remove Duplicates

Problem: Given a list with duplicate elements, return a new list containing only unique elements.

Idea: Cast the list to a set, which automatically filters out all duplicates. Then cast the set back to a list.

Code:

```
def remove_duplicates(arr):
    unique_set = set(arr)
    unique_list = list(unique_set)
    return unique_list
```

Problem 6.2: Array Intersection

Problem: Compute the intersection of two arrays (elements present in both).

Idea: Convert the first array to a set. Iterate over the second array, checking if elements exist in the set. Remove elements from the set after matching to avoid duplicates.

Code:

```
def array_intersection(arr1, arr2):
    set1 = set(arr1)
    result = []

    for number in arr2:
        if number in set1:
            result.append(number)
            set1.remove(number)

    return result
```

Problem 6.3: Missing Number

Problem: Given an array of n distinct numbers from ‘0‘ to ‘n‘, find the one missing number.

Idea: Insert all array elements into a set. Iterate from ‘0‘ to ‘n‘, checking if each number exists in the set.

Code:

```
def find_missing_number(arr):
    n = len(arr)
    num_set = set(arr)

    for i in range(n + 1):
        if i not in num_set:
            return i

    return None
```

Choosing the Right Data Structure

If you need to...	Use this Data Structure
Access data by an integer index	**List** (Array)
Constantly add/remove from ends	**Deque** (Doubly Linked)
Implement LIFO logic / Backtracking	**Stack** (List)
Process items sequentially (FIFO)	**Queue** (Deque)
Look up data by a custom key	**Dictionary** (Hash Map)
Ensure uniqueness / fast existence	**Set** (Hash Set)

Conclusion

Key Takeaways:

- High-level languages abstract memory management, but performance trade-offs ($O(1)$ vs $O(N)$) remain identical to lower-level languages.
- Proper structure selection drastically reduces algorithm complexity.